Verification Gates for Instinct: Pre-Integration Substrate-Resident Verification Suites Completing the Three Mutation Governance Instruments in the Coordination Knowledge Substrate Pattern

Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
May 7, 2026

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize **verification gates for instinct** — the pre-integration verification mechanism through which LLM versions and behavior changes must pass substrate-resident verification suites before adoption — as a standalone architectural commitment, completing the three mutation governance instruments per B1.13 (verification, routing per B2.04, pinning via high-stakes identification per B2.05) and closing the six-note B1.01 decomposition.

Abstract

Paper 2's instinct/reasoning separation (B1.01) commits the LLM-as-instinct layer and the substrate-as-reasoning layer to independent evolution under unified human governance. The three preceding decomposition notes formalize the layers themselves (B2.01, B2.02), the architectural test for separation (B2.03), the runtime routing patterns by which reasoning routes around or pins around instinct (B2.04), and the substrate-resident identification of which decisions count as high-stakes (B2.05). What remains is the *integration boundary* — the architectural pre-condition under which a new or changed LLM version becomes available for cell consultations at all. This note formalizes that pre-condition as **verification gates for instinct**: substrate-resident verification suites authored per A2.04, classified as Category 4 authoritative content per A2.46, recorded with provenance per A2.40, that an LLM version must pass before integration. The note states the mechanism precisely, distinguishes it from the unceremonious-update and informal-engineering-testing patterns common in conventional AI deployments, enumerates the operational verification mechanisms the architecture admits, articulates the inherited Paper 1 commitments, names the operational implications, identifies the limits, and gives a one-sentence operational test. The note closes the B1.01 decomposition by completing the three mutation governance instruments per B1.13.

1. Why verification-gates-for-instinct needs to be formalized as standalone

The instinct/reasoning separation per B1.01 places the LLM in the role of the instinct layer — a fast-pattern responder whose behavior is not authored by the deployment but consumed from upstream. B1.13 commits CKS deployments to govern instinct evolution through three architectural instruments operating in concert: **verification** (this note), **routing** (B2.04), and **pinning** (operationalized through the high-stakes identification of B2.05). The three together are what distinguishes CKS from biology's unceremonious

mutation-without-governance, and from the conventional AI deployment pattern in which an LLM provider releases a new version and downstream systems consume it without architectural intermediation.

The three instruments are not interchangeable. Routing per B2.04 governs *runtime* behavior and assumes that the LLM versions referenced by routing rules have already been authorized. Pinning via B2.05 governs *which decisions* the routing layer keeps on the reasoning path regardless of how capable instinct becomes — also at runtime. Neither addresses the question logically prior to both: under what architectural condition does a *new* LLM version (a vendor release, a new model variant, a fine-tuned instance, a replacement vendor under A6.03 forced migration, an upgraded substrate-platform host that changes LLM behavior) become eligible to appear in routing rules at all? Treating that question as deployment hygiene — engineers test, engineers decide — collapses it into the same informal-engineering category the rest of CKS holds at arm's length. The remedy is to name the verification gate as a standalone architectural commitment with substrate-resident content.

This note is the sixth and closing note in the B1.01 decomposition. With B2.01 (instinct layer), B2.02 (reasoning layer), B2.03 (architectural test), B2.04 (routing patterns), and B2.05 (high-stakes identification) already specified, B2.06 completes the three mutation governance instruments per B1.13 and closes the decomposition before Phase B2 turns to B1.02 in B2.07.

2. The architectural mechanism, precisely stated

A CKS deployment instantiates **verification gates for instinct** when, for every LLM version that becomes eligible for cell consultations, the version has first passed a substrate-resident verification suite under the rules the deployment authors. The mechanism has six operational components.

(a) Verification suites are substrate-resident authoritative content. The test cases against which an LLM version is checked, the expected outputs or output-shape constraints those test cases require, the pass-rate threshold at which the suite is considered passed, and the reviewer requirements determining who must approve the verification result are all substrate content under the standard authority architecture — categorically authoritative per A2.46 and inspectable per A2.01.

(b) Verification rules are authored by humans. Per A2.04, the rule-authoring commitment from Paper 1 extends to verification rules without modification. LLM-drafted verification suites subject to human authority before they take effect are admissible; LLM-committed verification rules outside human authority are not.

(c) The gate is pre-integration. Verification runs before the LLM version is referenced by any routing rule per B2.04. A version that has not passed verification is not addressable from any cell consultation path. Versions in flight — under verification but not yet through it — exist as substrate content in a known state but are not yet *integrated* in the architectural sense.

(d) Pass / fail / partial are the three architectural outcomes. A passed verification permits integration; the version becomes referenceable by routing rules. A failed verification prevents integration; the version is not used for cell consultations. A partial verification — passes on some test cases, fails on others — is a *governance event* rather than an architectural verdict: humans determine, under the deployment's rules, whether partial pass permits limited integration via narrowed routing rules, requires re-verification with revised suites, or triggers rejection.

(e) Verification events are recorded with provenance. Per A2.40's six provenance metadata fields, the substrate carries which LLM version was verified, against which suite version, when, with what pass-rate, by whom approved, and under which version of the verification rules. The provenance record is what makes verification history retraceable per A1.07.

(f) The mechanism covers behavior characteristics, not LLM internals. The suite tests the LLM version's behavior on representative inputs; it does not inspect model architecture, weights, or training data. Behavioral coverage typically includes consistency with the prior LLM version where applicable, pattern recognition on inputs from the deployment domain, response-format compatibility with cell processing logic, absence of regression on critical test cases, and presence of expected new capabilities for upgrade scenarios. What the suite covers in any given deployment is itself substrate content humans author.

The six components together define the architectural mechanism. A deployment that lacks any one — even if the others are robustly satisfied — does not implement verification gates for instinct in the CKS sense.

3. What makes verification-gates-for-instinct architecturally distinctive

Two patterns in the surrounding AI-deployment landscape are commonly conflated with verification gates and need to be named to keep the commitment precise.

Not unceremonious LLM updates. The most common pattern in conventional AI deployments is for the deployed LLM to update when the upstream provider releases a new version, with the deployment consuming the new version without architectural pre-condition. Some deployments add a configuration flag pinning to a specific version; others auto-upgrade. In neither case is the integration *gated* by a mechanism the deployment governs. Verification gates for instinct commit to the gate as architectural content under the deployment's authority.

Not informal engineering testing. A second pattern is that engineers maintain ad-hoc test scripts, run them when an upgrade arrives, and decide informally whether the new version is acceptable. This pattern produces verification-shaped *labor* but not verification-shaped *substrate*. The test scripts are infrastructure designed once and operated thereafter — antecedent to the deployment's substrate rather than part of it. The decision rules are tacit; the pass-rate threshold (if any) lives in engineering judgment. CKS commits the verification suites, the thresholds, the reviewer requirements, and the recorded pass-rates to substrate content under standard authority — not as deployment hygiene but as architectural primitive.

The substrate-resident character has direct consequences. Verification suites can be inspected per A2.01, modified per A2.02, and audited per A1.07. The verification rules themselves are subject to revision per A6.02; historical verification events recorded under prior rule versions remain retraceable through the A2.40 fourth field that names the rule version under which each event was recorded. Verification is also distinct from operational testing the system undergoes through other architectural tests — the A5.05 mediator-role test verifies architectural properties of LLM consultation, while verification gates per B2.06 verify behavior characteristics relevant to integration decisions. The two are independent: a deployment can pass A5.05 mediator-role verification while failing B2.06 behavioral verification on a new LLM version, or vice versa.

4. Operational verification mechanisms the architecture admits

The substrate-resident character does not specify a fixed mechanism shape. Within the six components of §2, deployments configure verification at the level of operational detail their context requires. Six operational mechanisms appear across CKS-coherent deployments.

Test-case substrate covering deployment domain. The suite holds representative inputs the deployment expects the LLM to handle — pattern-recognition tasks, response-format examples, edge cases observed in production, regression cases from prior incidents. The coverage is what the suite is authored to cover; it does not aspire to exhaustiveness.

Behavioral expectations specified per case. For each test case the suite carries either an expected output or a constraint on outputs (format, content, refusal-handling). The expectation form is itself substrate content — humans author what counts as a passing response per case.

Pass-rate thresholds defined per suite. The threshold at which a suite is considered passed (for example, ninety-five percent of test cases producing expected outputs) is substrate content authored under deployment governance. Different suites may carry different thresholds; high-stakes-decision suites per B2.05 typically carry stricter thresholds than exploratory suites.

Reviewer requirements per integration decision. Who approves verification results is substrate content. Some deployments require multiple reviewers; some require domain-specific reviewers; some delegate to authority-architecture roles operating at sub-Self scope. The requirement structure is itself authored under governance.

Suite versioning per A6.14. Verification suites evolve over deployment lifecycle through directed selection per B1.14: deployments add test cases as behavior issues emerge, retire cases as no longer representative, refine pass-rate thresholds as understanding grows. Suite versioning under A6.14 is what preserves retraceability of verification events across suite evolution.

Differential and canary verification. Two derived patterns appear within the substrate-resident framework: *differential verification* compares a new LLM version against the prior version on identical test cases, surfacing behavioral drift; *canary verification* permits gradual exposure of a new version under narrowed routing rules with automated monitoring before broader integration. Both are mechanism *shapes* the substrate-resident framework admits, not separate commitments.

The six together describe the operational room CKS deployments configure within. The architectural commitment is that whatever shape the verification takes, its content lives in substrate.

5. Inherited Paper 1 commitments

Verification gates for instinct sit on Paper 1's foundation in several specific ways. The **A2.04 rule authoring** commitment grounds verification rules as human-authored content. The **A2.46 Category 4** classification places verification suites in the authoritative-content space subject to standard authority. The **A2.40 provenance** commitment makes verification events retraceable across suite revisions. The **A1.01 human-governed** commitment keeps the three rights (inspect, modify, override) exercisable over verification configuration. The **A1.04 mediator role** is preserved: verification operates within the LLM-as-substrate-mediator commitment, gating which mediators are eligible rather than altering the mediator role itself. The **A1.05 tool-agnosticism** commitment underwrites verification's role in informed

vendor migration: a deployment switching LLM vendors under A6.03 forced migration runs the replacement vendor through the same verification suite, surfacing behavioral compatibility before integration. The **A2.62 bounded non-determinism** commitment is what verification's pass-rate thresholds operationalize — verification does not require deterministic LLM output but bounds the LLM's behavior on test cases within the categories of non-determinism Paper 1 admits. The **A1.07 path retraceability** commitment makes verification history a permanent property of the substrate.

None of these are new commitments; they are inherited references that verification gates instantiate at the integration boundary.

6. Operational implications

Six operational implications follow directly from the architectural mechanism.

Deployments **configure verification per operational requirements**: high-stakes domains identified per B2.05 carry stricter verification (stricter thresholds, more reviewers, broader test cases); exploratory uses may carry lighter verification. The configurability sits under standard authority.

Verification suites **evolve over deployment lifecycle**. As behavior issues surface, new test cases are added; as cases become unrepresentative, they are retired; as understanding grows, thresholds are refined. The evolution is itself substrate content with provenance per A2.40.

Verification **interacts with high-stakes identification** per B2.05: high-stakes decisions may carry separate verification suites with stricter pass-rate thresholds, ensuring versions used on the high-stakes path meet a sharper bar than versions used on routine paths.

Verification **interacts with vendor governance**. A6.03 forced migration to a replacement vendor triggers verification of the replacement; A6.09 vendor data-handling-policy changes may trigger re-verification of the existing version under altered conditions; A6.04 LLM consultation timeout patterns may indicate behavior shift warranting re-verification.

Verification **informs routing per B2.04**: passed verification is the architectural pre-condition for the new version to become referenceable by routing rules; routing rules may also be restricted in scope (canary patterns) when partial verification permits limited integration.

Verification is **operationally testable** per A5.16: a deployment's verification events replay deterministically under the recorded rule and suite versions, and the determinism contract per A1.10 holds at the verification layer as it does throughout the substrate.

7. Limits

Naming what verification gates for instinct *do not* commit to is what keeps the standalone framing precise.

Verification gates **do not verify LLM internals**. The suite tests behavior on inputs; it does not inspect weights, attention patterns, or training data. Internal verification of LLM model state is a different commitment that CKS does not make.

Verification gates **do not eliminate behavior shifts** — they provide an architectural framework to manage shifts at the integration boundary. Behavior on inputs outside the test cases may still differ between versions,

which is what bounded non-determinism per A2.62 makes explicit.

Verification suites **are not exhaustive**. They cover what they are authored to cover. Some behavior shifts will not appear in test cases until production use surfaces them; the directed-selection commitment per B1.14 is what allows suites to grow in response.

Verification **is not a one-time event**. It runs at integration and may re-run under multiple triggers — vendor changes per A6.03, policy changes per A6.09, anomaly patterns from A6.04, periodic review under deployment governance.

Verification results **do not bind future behavior**. Passing on the suite at integration time does not warrant identical behavior on inputs outside the suite at any later time. Verification is the integration gate; runtime monitoring is a separate operational concern.

Verification gates **do not replace operational tests** per A5.05 and the broader A5 suite. Operational tests verify architectural properties; verification gates verify behavior characteristics for integration decisions. The two coexist.

Verification operates **at the integration level**, not at the runtime level. Once a version is integrated, runtime use is governed by routing per B2.04 and pinning via B2.05; runtime behavior monitoring is operational concern distinct from verification gates.

Verification rules **can be revised** per A6.02 retroactivity. Historical verification events remain recorded under their original rule version per the A2.40 fourth field; revisions affect future verification events, not the historical record.

8. Operational test

A CKS deployment instantiates the **verification gates for instinct** commitment if and only if every LLM version eligible for cell consultations has, prior to that eligibility, passed a substrate-resident verification suite authored under the deployment's rules per A2.04, with the pass-rate threshold and reviewer approval recorded as substrate provenance per A2.40, with the suite content classified as authoritative substrate per A2.46, with no LLM operation, vendor policy, or runtime middleware able in principle to bypass the gate, and with verification rule revisions handled retroactively per A6.02 such that historical verification events remain retraceable under their original rule version.

A deployment that fails any clause may have useful integration practices but does not implement verification gates for instinct in the CKS sense.

9. Why naming verification gates as standalone matters; closing the B1.01 decomposition

Treating verification as deployment hygiene makes mutation governance per B1.13 architecturally incomplete. Without a substrate-resident integration gate, the mutation pillar of the three-instrument structure (verification, routing per B2.04, pinning via B2.05) is only partially specified — routing and pinning operate at runtime and presuppose that the versions they reference have been validated, but the validation itself is left to ad-hoc engineering practice. Naming verification gates as standalone is what makes the three instruments architecturally complete. With verification at the integration boundary, routing at the runtime boundary, and pinning at the high-stakes runtime boundary, CKS deployments hold a complete

mutation governance architecture distinct from biology's mutation-without-governance and from conventional AI deployments' integration-without-architectural-intermediation.

This note closes the six-note B1.01 decomposition. With B2.01 specifying the instinct layer's operational definition, B2.02 the reasoning layer's, B2.03 the architectural test for separation, B2.04 instinct-routing patterns, B2.05 high-stakes decision identification, and B2.06 verification gates, the B1.01 commitment to instinct/reasoning separation as independently-evolving layers under unified human governance is decomposed into the operational variants any CKS-coherent deployment instantiates. Phase B2 continues with B2.07, opening the B1.02 three-level structure (cell, aspect, Self with relational role membership) decomposition.

Subsequent work that adopts, extends, composes with, or argues against the verification-gates-for-instinct commitment should use the term in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Verification Gates for Instinct: Pre-Integration Substrate-Resident Verification Suites Completing the Three Mutation Governance Instruments in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.